МИРЭА- Российский технологический университет

Институт перспективных технологий и индустриального программирования

Дисциплина

«Алгоритмы машинного обучения для решения прикладных задач»

Семестр 1

Практическое занятие №9

Практическая работа кластеризация k-means, k-means++.

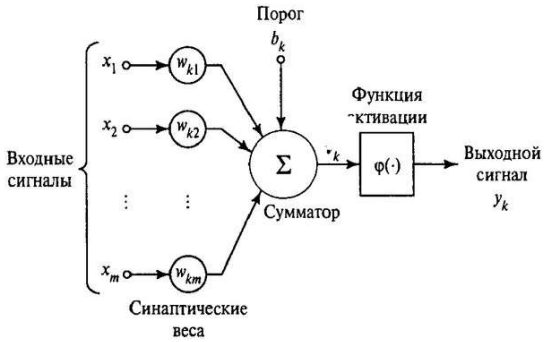
Доцент кафедры индустриального программирования
к. ф.-м. н.
Осипов Алексей Викторович

Москва, 2026 г.

Создание нейронной сети на Torch

In [1]:
```
import torch
import numpy as np
```

Нейрон



$$s = \sum_i (w_i x_i) + b$$

In [4]:
```
# Создание класса нейрон. weights - веса связей, bias - порог, input - вектор

class Neuron:
  def __init__(self, weights, bias):
    self.weights = weights
    self.bias = bias

  def forward(self, input):
    return torch.dot(self.weights, input) + self.bias # dot - скалярное произв
```

In [5]:
```
inputs = torch.tensor([1.0, 2.0, 3.0, 2.5])

weights = torch.tensor([0.2, 0.8, -0.5, 1.0])
bias = 2.0

neuron = Neuron(weights, bias)
neuron.forward(inputs)
```

Out[5]:
```
tensor(4.8000)
```

Слой

In [ ]:
```
<img src="img/Layer.png" width=300, heigth=300>
```

In [6]:
```
class Layer:
  def __init__(self, weights, biases):
    self.weights = weights
    self.biases = biases

  def forward(self, input):
    return torch.mv(self.weights, input) + self.biases #mv - произведение векто
```

In [7]:
```
inputs = torch.tensor([1.0, 2.0, 3.0, 2.5])
weights = torch.tensor([[0.2, 0.8, -0.5, 1.0],
           [0.5, -0.91, 0.26, -0.5],
           [-0.26, -0.27, 0.17, 0.87]])

biases = torch.tensor([2.0, 3.0, 0.5])

layer = Layer(weights, biases)
layer.forward(inputs)
```

Out[7]:
```
tensor([4.8000, 1.2100, 2.3850])
```

Слой (batch)

обработка партиями

In [10]:
```
class Layer:
  def __init__(self, weights, biases):
    self.weights = weights
    self.biases = biases

  def forward(self, inputs):
    return inputs @ self.weights.T + self.biases # провизведение матриц
```

In [12]:
```
inputs1 = torch.tensor([[1, 2, 3, 2.5], [2, 5, -1, 2], [-1.5, 2.7, 3.3, -0.8]])
weights1 = torch.tensor([[0.2, 0.8, -0.5, 1.0],
           [0.5, -0.91, 0.26, -0.5],
           [-0.26, -0.27, 0.17, 0.87]])
biases1 = torch.tensor([2.0, 3.0, 0.5])

weights2 = torch.tensor([[.1, -.14, .5], [-.5, .12, -.33], [-.44, .73, -.13]])
biases2 = torch.tensor([-1, 2, -.5])

layer1 = Layer(weights1, biases1)
hidden_input = layer1.forward(inputs1)

layer2 = Layer(weights2, biases2)
output = layer2.forward(hidden_input)
output
```

Out[12]:
```
tensor([[ 0.5031, -1.0419, -2.0388],
        [ 0.2434, -2.7332, -5.7633],
        [-0.9931,  1.4125, -0.3565]])
```

Слой (случ. иниц)

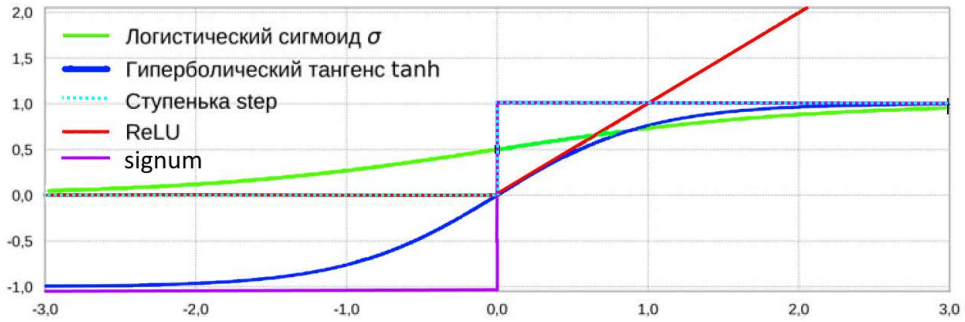
In [14]:
```
class Layer:
  def __init__(self, n_inputs, n_neurons):
    self.weights = torch.randn(n_inputs, n_neurons)
    self.biases = torch.zeros(n_neurons)

  def forward(self, inputs):
    return inputs @ self.weights + self.biases
```

In [15]:
```
inputs1 = torch.tensor([[1, 2, 3, 2.5], [2, 5, -1, 2], [-1.5, 2.7, 3.3, -0.8]])
layer = Layer(4, 6)
out = layer.forward(inputs1)
out
```

Out[15]:
```
tensor([[-5.4866e-03,  8.6038e+00,  8.1644e-01, -9.8074e+00,  7.0898e+00,
         -1.2581e-01],
        [-5.8837e+00,  1.3764e+01,  2.2832e+00,  3.7342e+00,  1.1002e+01,
         -8.5991e-01],
        [-2.9182e+00,  3.6327e+00, -1.6558e+00, -2.3815e+00,  4.7822e+00,
         -5.3641e+00]])
```

Функции активации



Функция активации ReLU:

$$f(x) = \begin{cases} 0 & x < 0 \\ x & x \geqslant 0 \end{cases}$$

Функцию активации softmax:

$$f_i(\vec{x}) = \frac{e^{x_i}}{\sum_{j=1}^{J} e^{x_j}}$$

In [16]:
```
# RELU
class ReLU:
  def forward(self, inputs):
    return inputs.clip(min=0)    # заменить отрицательные числа нулями
```

In [17]:
```
ReLU().forward(out)
```

Out[17]:
```
tensor([[ 0.0000,  8.6038,  0.8164,  0.0000,  7.0898,  0.0000],
        [ 0.0000, 13.7638,  2.2832,  3.7342, 11.0023,  0.0000],
        [ 0.0000,  3.6327,  0.0000,  0.0000,  4.7822,  0.0000]])
```

In [18]:
```
class Softmax:
  def forward(self, inputs):
    exp = torch.exp(inputs)
    probs = exp / exp.sum(dim=1, keepdim=True)
    return probs
```

In [19]:
```
Softmax().forward(out)
```

Out[19]:
```
tensor([[1.4942e-04, 8.1914e-01, 3.3990e-04, 8.2697e-09, 1.8024e-01, 1.3248e-
04],
        [2.7576e-09, 9.4051e-01, 9.7135e-06, 4.1451e-05, 5.9437e-02, 4.1913e-
07],
        [3.4299e-04, 2.4006e-01, 1.2121e-03, 5.8667e-04, 7.5777e-01, 2.9721e-
05]])
```

Функции потерь

Функция потерь MSE:

$$\mathbf{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2.$$

Функция потерь Categorical Cross-Entropy:

$$L_i = -\sum_j y_{i,j} log(\hat{y}_{i,j})$$

In [24]:
```
inputs1 = torch.tensor([[1, 2, 3, 2.5], [2, 5, -1, 2], [-1.5, 2.7, 3.3, -0.8]])
layer = Layer(4, 1)
out = layer.forward(inputs1)
out
```

Out[24]:
```
tensor([[ 2.9493],
        [-0.2413],
        [ 9.7527]])
```

In [25]:
```
# Среднеквадратичная ошибка
class MSELoss:
  def forward(self, y_pred, y_true):
    return torch.mean(torch.pow(out - y_true, 2))
```

In [26]:
```
y_true = torch.tensor([2, 3, 4]).view(-1, 1)
MSELoss().forward(out, y_true)
```

Out[26]:
```
tensor(14.8337)
```

In [28]:
```
# Кроссэнтропия
class CategoricalCrossentropyLoss:
  def forward(self, y_pred, y_true):
    correct_conf = y_pred.gather(dim=1, index=y_true.view(-1, 1))
    neglog = -torch.log(correct_conf)
    return torch.mean(neglog)
```

In [29]:
```
inputs1 = torch.tensor([[1, 2, 3, 2.5], [2, 5, -1, 2], [-1.5, 2.7, 3.3, -0.8]])
layer = Layer(4, 3)
out = layer.forward(inputs1)
out = Softmax().forward(out)
out
```

Out[29]:
```
tensor([[9.9897e-01, 5.8893e-05, 9.7537e-04],
        [1.7042e-03, 9.9828e-01, 1.1649e-05],
        [6.2678e-01, 3.7259e-01, 6.2787e-04]])
```

In [30]:
```
y_true = torch.tensor([1, 0, 0])
CategoricalCrossentropyLoss().forward(out, y_true)
```

Out[30]:
```
tensor(5.5272)
```

Оптимизация SGD (стохастический градиентный спуск)

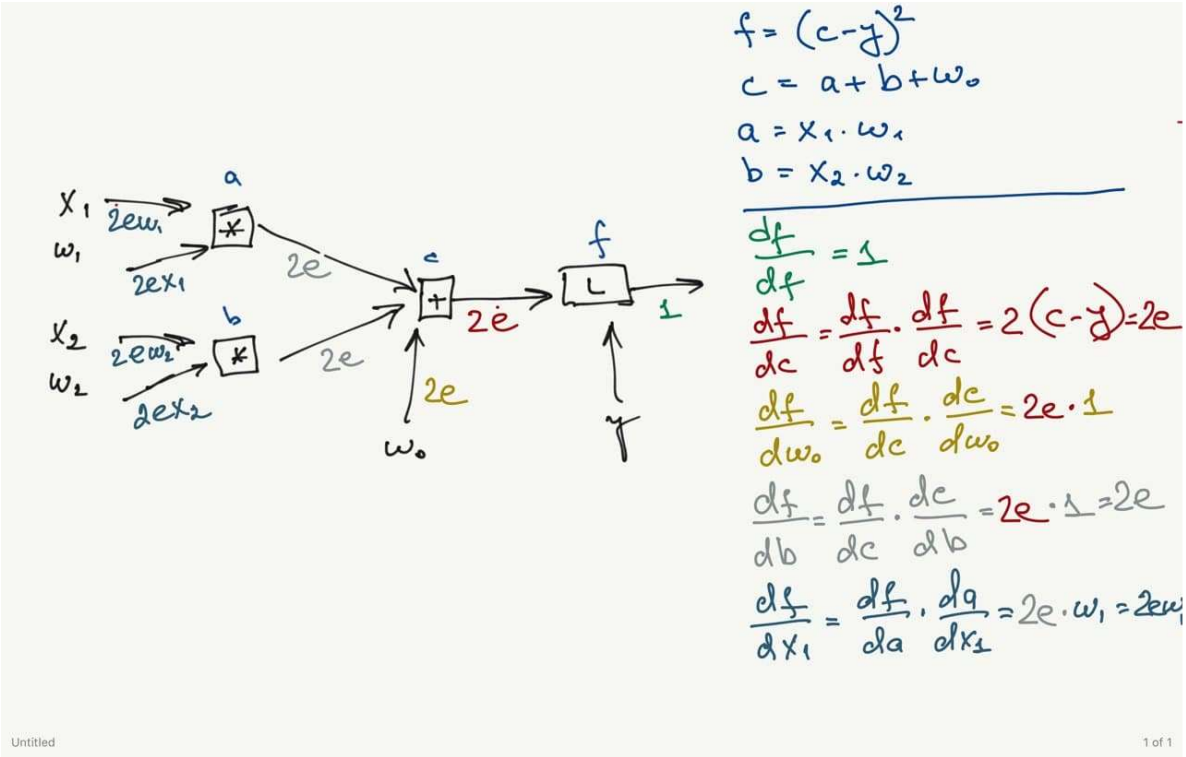
Модифицируем класс Neuron

Сделаем так, чтобы веса нейрона инициализировались из стандартного нормального распределения:

In [41]:
```
# Сформируем случайную задачу регркессии
from sklearn.datasets import make_regression

X, y, coef = make_regression(n_features=4, n_informative=4, coef=True, bias=0.
X = torch.from_numpy(X).to(dtype=torch.float32)
y = torch.from_numpy(y).to(dtype=torch.float32)
```

Граф вычислений для решения задачи:



In [40]:
```
class Neuron:
  def __init__(self, n_inputs):
    self.weights = torch.randn(n_inputs)
    self.bias = torch.randn(1)

  def forward(self, input):
    self.input = input
    return torch.dot(self.weights, input) + self.bias

  def backward(self, dvalue):
    self.dweights = dvalue * self.input
    self.dinput = dvalue * self.weights
    self.dbias = dvalue


class Loss:    #класс вычисления ошибок
  def forward(self, y_pred, y_true):
    return (y_pred - y_true) ** 2

  def backward(self, y_pred, y_true):
    self.dinput = 2 * (y_pred - y_true)
```

In [52]:
```
neuron = Neuron(4)
loss = Loss()

lr = 0.1

losses = []
for epoch in range(100):
  for x_example, y_example in zip(X, y):
    y_pred = neuron.forward(x_example)
    l = loss.forward(y_pred, y_example)
    losses.append(l)

    loss.backward(y_pred, y_example)
    neuron.backward(loss.dinput)

    neuron.weights -= lr * neuron.dweights
    neuron.bias -= lr * neuron.dbias
```

In [46]:
```
neuron.weights, neuron.bias
```

Out[46]:
```
(tensor([57.7219,  1.2114,  3.5078, 24.1808]), tensor([0.5000]))
```

Используем пакетный градиентный спуск (batch SGD + neuron)

In [53]:
```
class Neuron:
  def __init__(self, n_inputs):
    self.weights = torch.randn(n_inputs, requires_grad=False)
    self.bias = torch.tensor(0.0, requires_grad=False)

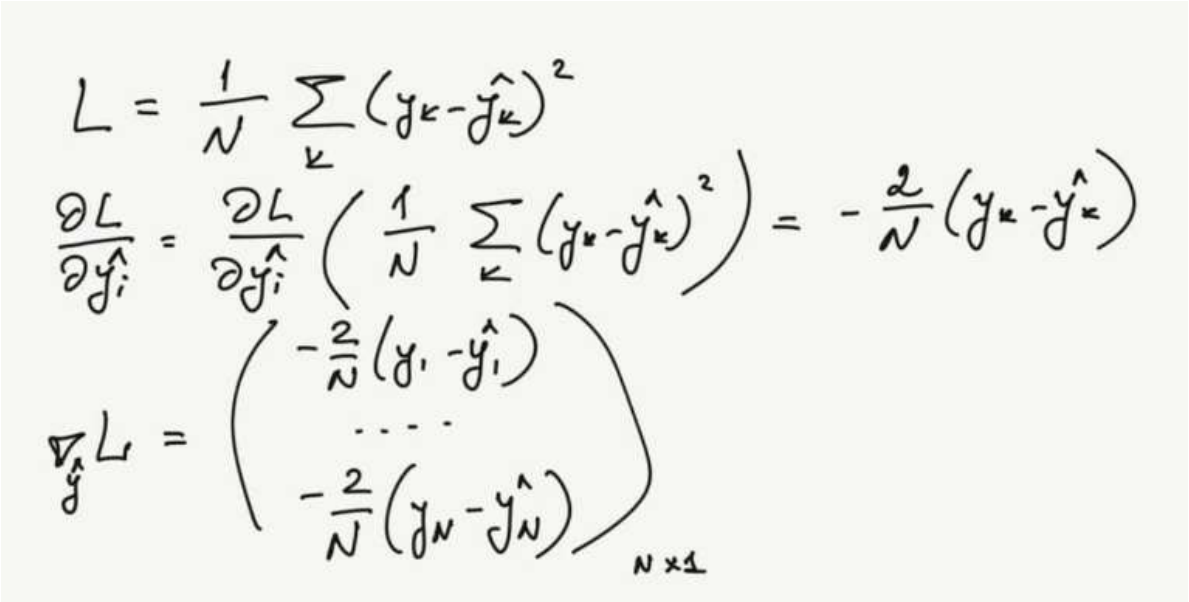
  def forward(self, inputs):
    self.inputs = inputs
    return X @ self.weights.T + self.bias

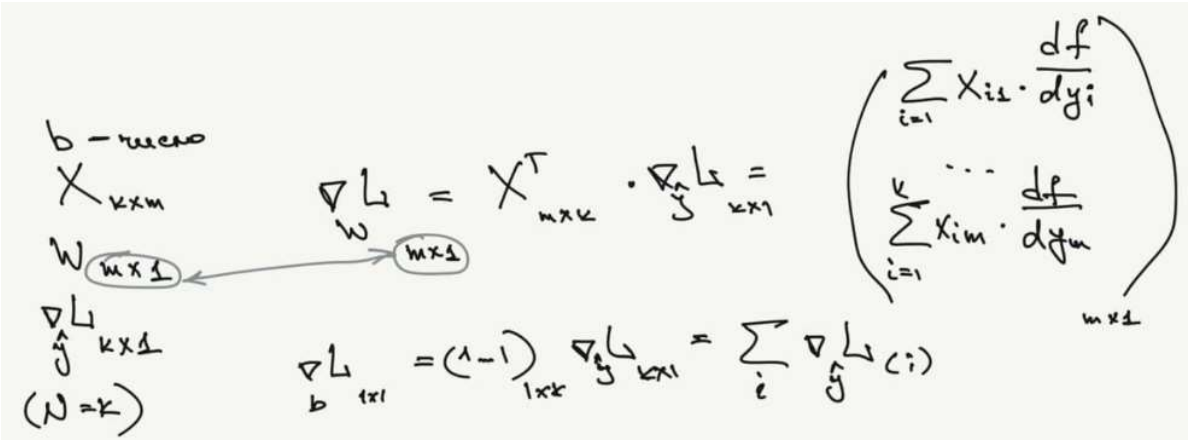
  def backward(self, dvalues):
    with torch.no_grad():
      self.dweights = self.inputs.T.mv(dvalues)
      self.dbiases = dvalues.sum()


class MSELossNeuron:
  def forward(self, y_pred, y_true):
    return torch.mean((y_pred - y_true) ** 2)

  def backward(self, y_pred, y_true):
    with torch.no_grad():
      self.dinputs = 2 * (y_pred - y_true) / len(y_pred)
```

Вычисления для решения задачи:





In [60]:
```
from sklearn.datasets import make_regression

X, y, coef = make_regression(n_features=4, n_informative=4, coef=True, bias=0.
X = torch.from_numpy(X).to(dtype=torch.float32)
y = torch.from_numpy(y).to(dtype=torch.float32)
```

In [61]:
```
neuron = Neuron(4)
loss = MSELossNeuron()

lr = 0.1

for epoch in range(100):
    y_pred = neuron.forward(X)
    l = loss.forward(y_pred, y)
    loss.backward(y_pred, y)
    neuron.backward(loss.dinputs)

    neuron.weights -= lr * neuron.dweights
    neuron.bias -= lr * neuron.dbiases
print(f'epoch {epoch} mean loss {l.mean()}')
```

```
epoch 99 mean loss 3.637296575043081e-10
```

In [62]:
```
neuron.weights, coef
```

Out[62]:
```
(tensor([34.1417, 62.3545, 60.4058, 52.2198]),
 array([34.14168559, 62.35449431, 60.40579793, 52.21978419]))
```

Разные варианты веализации рейросети

In [64]:
```
class Dense:
  def __init__(self, n_inputs, n_neurons):
    self.weights = torch.randn(n_inputs, n_neurons, requires_grad=False)
    self.biases = torch.zeros(n_neurons, requires_grad=False)

  def forward(self, inputs):
    self.inputs = inputs
    self.output = self.inputs @ self.weights + self.biases
    return self.output

  def backward(self, dvalues):
    with torch.no_grad():
      self.dweights = self.inputs.T @ dvalues
      self.dbiases = torch.sum(dvalues, dim=0)
      self.dinputs = dvalues @ self.weights.T

class Activation_ReLU:
  def forward(self, inputs):
    self.inputs = inputs
    self.output = inputs.clip(min=0)
    return self.output

  def backward(self, dvalues):
    with torch.no_grad():
      self.dinputs = dvalues.clone()
      self.dinputs[self.inputs <= 0] = 0

class Activation_Linear:
  def forward(self, inputs):
    self.inputs = inputs
    self.output = inputs

  def backward(self, dvalues):
    with torch.no_grad():
      self.dinputs = dvalues.clone()

class Loss:
  def calculate(self, output, y):
    sample_losses = self.forward(output, y)
    data_loss = sample_losses.mean()
    return data_loss

class MSELoss(Loss):
  def forward(self, y_pred, y_true):
    sample_losses = torch.mean((y_true - y_pred) ** 2, dim=1)
    return sample_losses

  def backward(self, dvalues, y_true):
    with torch.no_grad():
      self.dinputs = -2 * (y_true - dvalues) / len(dvalues)
```

In [65]:
```
from sklearn.datasets import make_regression

X, y, coef = make_regression(n_features=4, n_informative=4, coef=True, bias=0.
X = torch.from_numpy(X).to(dtype=torch.float32)
y = torch.from_numpy(y).to(dtype=torch.float32).view(-1, 1)
```

In [66]:
```
fc1 = Dense(4, 1)
loss = MSELoss()
lr = 0.1

for epoch in range(100):
  out = fc1.forward(X)
  data_loss = loss.calculate(out, y.view(-1, 1))
  print(f'epoch {epoch} mean loss {data_loss}')
  loss.backward(out, y.view(-1, 1))
  fc1.backward(loss.dinputs)

  fc1.weights -= lr * fc1.dweights
  fc1.biases -= lr * fc1.dbiases
```

```
epoch 0 mean loss 18606.173828125
epoch 1 mean loss 10852.302734375
epoch 2 mean loss 6439.59423828125
epoch 3 mean loss 3883.696044921875
epoch 4 mean loss 2377.51220703125
epoch 5 mean loss 1475.1981201171875
epoch 6 mean loss 926.3163452148438
epoch 7 mean loss 587.7548217773438
epoch 8 mean loss 376.3168640136719
epoch 9 mean loss 242.8217010498047
epoch 10 mean loss 157.73350524902344
epoch 11 mean loss 103.05297088623047
epoch 12 mean loss 67.6644058227539
epoch 13 mean loss 44.6216926574707
epoch 14 mean loss 29.538488388061523
epoch 15 mean loss 19.62017250061035
epoch 16 mean loss 13.071903228759766
epoch 17 mean loss 8.733251571655273
epoch 18 mean loss 5.849470138549805
epoch 19 mean loss 3.9271962642669678
epoch 20 mean loss 2.642458200454712
epoch 21 mean loss 1.7817388772964478
epoch 22 mean loss 1.2037780284881592
epoch 23 mean loss 0.8148624300956726
epoch 24 mean loss 0.5526217818260193
epoch 25 mean loss 0.3754558563232422
epoch 26 mean loss 0.2555404603481293
epoch 27 mean loss 0.1742214560508728
epoch 28 mean loss 0.11898254603147507
epoch 29 mean loss 0.08139253407716751
epoch 30 mean loss 0.055771637707948685
epoch 31 mean loss 0.03827638551592827
epoch 32 mean loss 0.02631310746073723
epoch 33 mean loss 0.018116768449544907
epoch 34 mean loss 0.012492692098021507
epoch 35 mean loss 0.008627964183688164
epoch 36 mean loss 0.005968089681118727
epoch 37 mean loss 0.004134447779506445
epoch 38 mean loss 0.0028680595569312572
epoch 39 mean loss 0.001992202829569578
epoch 40 mean loss 0.0013861502520740032
epoch 41 mean loss 0.0009657860500738025
epoch 42 mean loss 0.0006737485527992249
epoch 43 mean loss 0.0004705431347247213
epoch 44 mean loss 0.00032891248702071607
epoch 45 mean loss 0.00023035566846374422
epoch 46 mean loss 0.00016145542031154037
epoch 47 mean loss 0.00011333516158629209
epoch 48 mean loss 7.961018854985014e-05
epoch 49 mean loss 5.604670514003374e-05
epoch 50 mean loss 3.948171797674149e-05
epoch 51 mean loss 2.7823592972708866e-05
epoch 52 mean loss 1.9622166291810572e-05
epoch 53 mean loss 1.3849032256985083e-05
epoch 54 mean loss 9.777235391084105e-06
epoch 55 mean loss 6.901186225150013e-06
epoch 56 mean loss 4.905240075459005e-06
```

```
epoch 57 mean loss 3.4738391150312964e-06
epoch 58 mean loss 2.4654207209096057e-06
epoch 59 mean loss 1.7564650534040993e-06
epoch 60 mean loss 1.2465374084058567e-06
epoch 61 mean loss 8.840727900860657e-07
epoch 62 mean loss 6.283231641646125e-07
epoch 63 mean loss 4.46199578618689e-07
epoch 64 mean loss 3.184001116096624e-07
epoch 65 mean loss 2.288361997671018e-07
epoch 66 mean loss 1.6340331399078423e-07
epoch 67 mean loss 1.1672965882780773e-07
epoch 68 mean loss 8.16602820918888e-08
epoch 69 mean loss 5.798530366973864e-08
epoch 70 mean loss 4.170296818983843e-08
epoch 71 mean loss 3.005733972827329e-08
epoch 72 mean loss 2.102363616529601e-08
epoch 73 mean loss 1.557245177252753e-08
epoch 74 mean loss 1.1067509575468648e-08
epoch 75 mean loss 7.959320313943863e-09
epoch 76 mean loss 6.356813742058755e-09
epoch 77 mean loss 4.62312277349497e-09
epoch 78 mean loss 3.361429579129549e-09
epoch 79 mean loss 2.4906727791318417e-09
epoch 80 mean loss 1.8545998070607084e-09
epoch 81 mean loss 1.316910136495153e-09
epoch 82 mean loss 1.2028326112911714e-09
epoch 83 mean loss 8.751928071148996e-10
epoch 84 mean loss 7.251962363952202e-10
epoch 85 mean loss 6.936304863813803e-10
epoch 86 mean loss 6.508177885500288e-10
epoch 87 mean loss 6.114541650781291e-10
epoch 88 mean loss 6.11952932771942e-10
epoch 89 mean loss 6.660358375931708e-10
epoch 90 mean loss 6.713063438468225e-10
epoch 91 mean loss 6.714078737424245e-10
epoch 92 mean loss 6.717689737811838e-10
epoch 93 mean loss 6.713269384839293e-10
epoch 94 mean loss 6.700319188368553e-10
epoch 95 mean loss 6.701168508982391e-10
epoch 96 mean loss 6.696362908620301e-10
epoch 97 mean loss 6.697216115014726e-10
epoch 98 mean loss 6.700262566994297e-10
epoch 99 mean loss 6.70112021428082e-10
```

In [67]:
```
fc1.weights, coef
```

Out[67]:
```
(tensor([[74.6948],
         [ 6.0301],
         [55.5272],
         [86.9832]]),
 array([74.69482344,  6.03007611, 55.52723636, 86.98324043]))
```

In [68]:
```
from sklearn.datasets import make_regression

X, y, coef = make_regression(n_features=4, n_informative=4, coef=True, bias=0.
X = torch.from_numpy(X).to(dtype=torch.float32)
y = torch.from_numpy(y).to(dtype=torch.float32).view(-1, 1)
```

In [70]:
```
X = torch.linspace(-1, 1, 100).view(-1, 1)
y = X.pow(2) + 0.2 * torch.rand(X.size())
```

In [71]:
```
fc1 = Dense(1, 10)
relu1 = Activation_ReLU()
fc2 = Dense(10, 1)

loss = MSELoss()
lr = 0.02

ys = []
for epoch in range(2001):
  out = fc1.forward(X)
  out = relu1.forward(out)
  out = fc2.forward(out)


  data_loss = loss.calculate(out, y)
  if epoch % 200 == 0:
    print(f'epoch {epoch} mean loss {data_loss}')
    ys.append(out)
  loss.backward(out, y)
  fc2.backward(loss.dinputs)
  relu1.backward(fc2.dinputs)
  fc1.backward(relu1.dinputs)

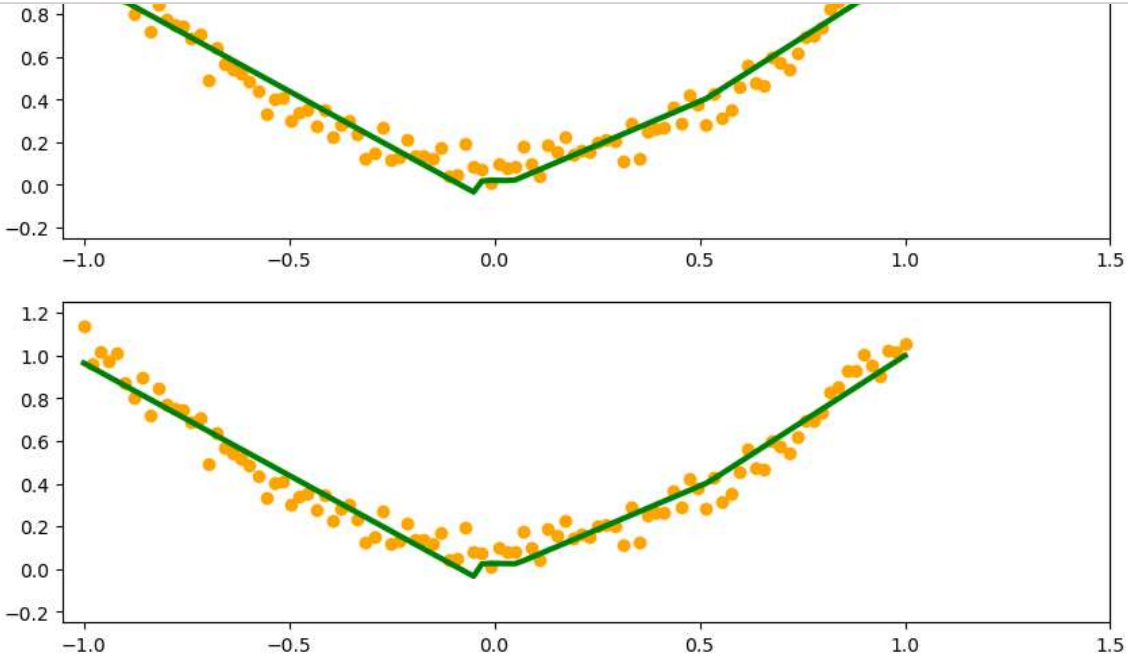
  fc1.weights -= lr * fc1.dweights
  fc1.biases -= lr * fc1.dbiases

  fc2.weights -= lr * fc2.dweights
  fc2.biases -= lr * fc2.dbiases
```

```
epoch 0 mean loss 1.4763978719711304
epoch 200 mean loss 0.008565408177673817
epoch 400 mean loss 0.006872948259115219
epoch 600 mean loss 0.006524837110191584
epoch 800 mean loss 0.006321059539914131
epoch 1000 mean loss 0.006160559598356485
epoch 1200 mean loss 0.006019381806254387
epoch 1400 mean loss 0.005901716183871031
epoch 1600 mean loss 0.00579681945964694
epoch 1800 mean loss 0.005711197853088379
epoch 2000 mean loss 0.005630379542708397
```

In [72]:
```
import matplotlib.pyplot as plt

fig, axs = plt.subplots(len(ys), 1, figsize=(10, 40))
for ax, y_ in zip(axs, ys):
  ax.scatter(X.numpy(), y.numpy(), color = "orange")
  ax.plot(X.numpy(), y_.numpy(), 'g-', lw=3)
  ax.set_xlim(-1.05, 1.5)
  ax.set_ylim(-0.25, 1.25)
```



In [ ]: